BÀI TẬP THỰC HÀNH LẬP TRÌNH BLOCKCHAIN

Xây Dựng Dịch Vụ API Sinh Địa Chỉ Thanh Toán Động (Dynamic Merchant Address Service) Dùng HD Wallet (BIP44)

1. BỐI CẢNH BÀI TOÁN

Trong hệ thống cổng thanh toán e-commerce phi tập trung, nếu thương nhân chỉ dùng **một địa chỉ ví duy nhất** để nhận tiền từ tất cả khách hàng, đối thủ hoặc bên thứ ba có thể dễ dàng tra cứu sổ cái công khai để biết toàn bộ doanh thu và lịch sử đơn hàng của cửa hàng.

Để bảo vệ quyền riêng tư, bạn được giao nhiệm vụ phát triển một **Dịch vụ Backend (REST API)** chạy trên cổng **5000**. Dịch vụ này sẽ sử dụng chuỗi bí mật **Mnemonic (Seed Phrase)** gốc của thương nhân để tự động tạo ra một **địa chỉ Ethereum hoàn toàn mới và duy nhất** cho mỗi lần khách hàng nhấn thanh toán đơn hàng.

2. YÊU CẦU KỸ THUẬT

Sinh viên cần khởi tạo một dự án Node.js/Express (app.js) và lập trình hoàn chỉnh các yêu cầu sau:

Yêu cầu 1: Khởi tạo môi trường & Thư viện

- Đăng ký và nạp các gói phụ thuộc bắt buộc: express, cors, bip39, hdkey, ethereumjs-util.
- Thiết lập cấu hình CORS cho phép ứng dụng Web Frontend (React.js chạy ở Port 3000) gọi API mà không bị chặn.

Yêu cầu 2: Quản lý Chuỗi Seed & Khóa Gốc (Master Node)

- Sử dụng thư viện bip39 để chuyển đổi chuỗi Mnemonic 12 từ thành chuỗi seed nhị phân.
- Sử dụng hdkey để khởi tạo nút ví gốc (root) từ seed vừa tạo.

Yêu cầu 3: Lập trình Route API GET /api/getMAddress

- Xây dựng đường dẫn phân cấp BIP44 cho Ethereum: path = "m/44'/60'/0'/0/" + pathid
- Dùng hàm root.derive(path) để lấy nút ví tương ứng.
- Trích xuất Khóa công khai (privateToPublic), tạo địa chỉ Hex (publicToAddress) và chuyển đổi sang định dạng chuẩn Checksum Address (toChecksumAddress).

- Quản lý chỉ số tự tăng: Khởi tạo pathid = 0. Tự động tăng pathid++ sau mỗi lần gọi API. Reset pathid = 0 nếu pathid >= 100.

3. DỮ LIỆU MẪU & INPUT / OUTPUT ĐỂ TEST

Input (Cấu hình trên Server):
• Chuỗi Mnemonic gốc: "share confirm story grocery check soon cool priority doctor cruise subway provide"
• Server Port: 5000

Output Mong Đợi khi Gọi API:

Lần gọi 1 (GET http://localhost:5000/api/getMAddress):
• Đường dẫn BIP44 sử dụng: m/44'/60'/0'/0/0
• Response JSON: {"MAddress": "0x3897B0FA54F0CD211e4fFDd1A48805F04336c285"}
Lần gọi 2 (GET http://localhost:5000/api/getMAddress):
• Đường dẫn BIP44 sử dụng: m/44'/60'/0'/0/1
• Response JSON: {"MAddress": "0x3150247FE43C651DC59A41eEf5ee9bCEdA6E72EB"}

4. MÃ NGUỒN GỢI Ý (STARTER SKELETON CODE)

Sinh viên điền mã nguồn hoàn chỉnh vào các vị trí // TODO dưới đây:

```
const express = require("express");
const cors = require("cors");
const bip39 = require("bip39");
const hdkey = require("hdkey");
const ethUtil = require("ethereumjs-util");

const app = express();
app.use(cors());

// Mnemonic seed phrase dùng chung cho cửa hàng
const mnemonic = "share confirm story grocery check soon cool
priority doctor cruise subway provide";
var pathid = 0; // Biến đếm chỉ số địa chỉ động

app.get("/api/getMAddress", async function (req, res) {
```

```
  try {
    // TODO 1: Chuyển mnemonic thành binary seed sử dụng bip39

    // TODO 2: Tạo nút master key (root) từ seed sử dụng hdkey

    // TODO 3: Tạo chuỗi derivation path chuẩn BIP44 với biến
pathid

    // TODO 4: Dẫn xuất child node từ root theo path vừa tạo

    // TODO 5: Tính toán Public Key -> Ethereum Address Hex ->
Checksum Address

    // TODO 6: Tăng biến pathid (Reset về 0 nếu pathid >= 100)

    // TODO 7: Trả về JSON chứa MAddress

  } catch (error) {
    res.status(500).send({ error: error.message });
  }
});

const PORT = 5000;
app.listen(PORT, () => {
  console.log(`Dynamic Merchant Address Service running on
port ${PORT}`);
});
```